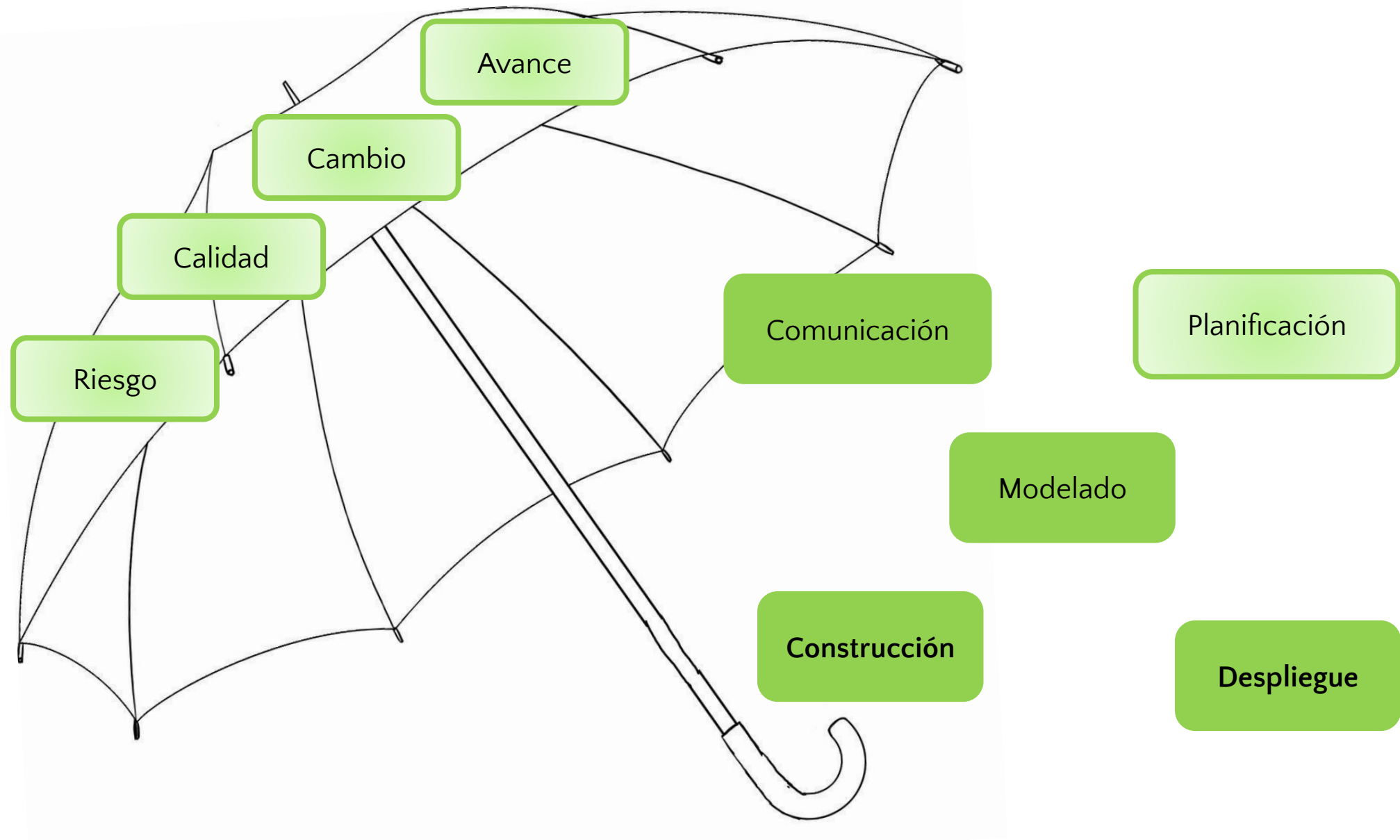


Introducción a la ingeniería de software





1

Implementación

Codificación. Documentación.



Implementación de una solución

Codificación del software

- Múltiples lenguajes de programación
- Varias personas involucradas en el proceso
- Grupos de trabajo: cooperación y coordinación

Tu haces la introducción, tu la conclusión, yo la bibliografía **y mañana unimos todo eso en la universidad.**





Implementación de una solución

Codificación del software

- Múltiples lenguajes de programación
- Varias personas involucradas en el proceso
- Grupos de trabajo: cooperación y coordinación

Se definen estándares de programación

- Estilo, formato: código y documentación





Pautas para la programación

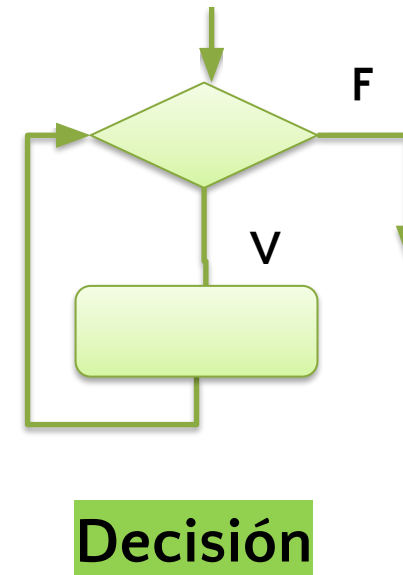
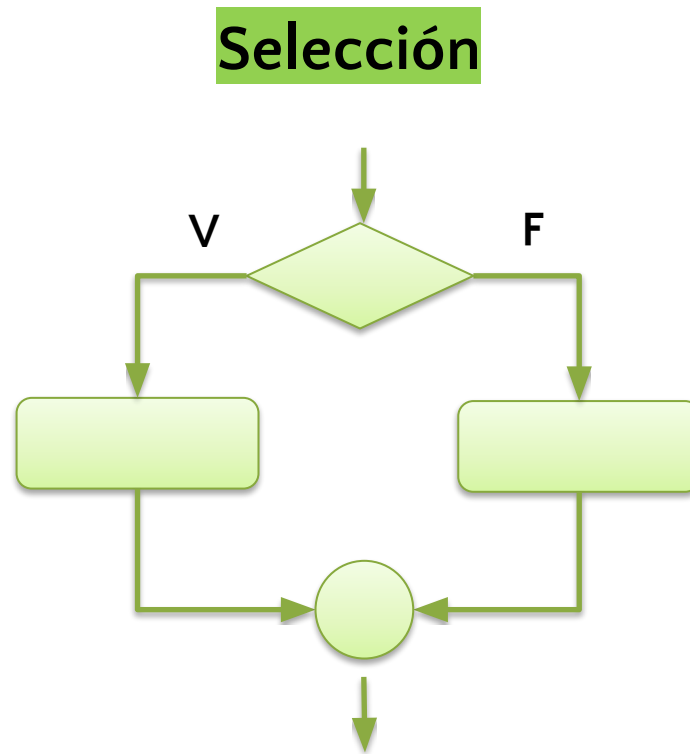
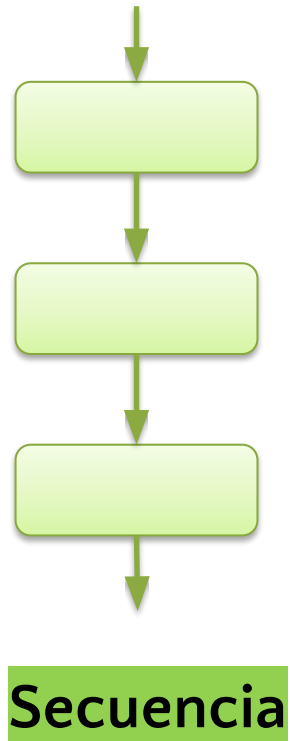
A partir del diseño del componente el programador tiene libertad para aplicar su creatividad en la implementación.

Cada componente de un programa implica al menos tres aspectos principales:

- 1-Estructuras de control
- 2-Algoritmos
- 3-Estructuras de datos



1-Estructuras de control





1-Estructuras de control

Determinadas por la estructura misma del código.

Debe poder leerse fácilmente un componente de lo general a lo particular.

Modularidad: también aplicarla al código

- Facilita el mantenimiento y la reutilización
- Generalizar todo lo posible sin afectar la performance o la facilidad de comprensión



2-Algoritmos

Se debe tener en cuenta:

- **Tiempo** de ejecución
- Costo
 - de **escritura**
 - de **prueba**: en tiempo y en dificultad de construcción de los casos de prueba
 - de **modificación** cuando sea necesario

No sacrificar claridad y corrección por velocidad de ejecución

3-Estructuras de datos



Búsqueda por autor: **Tolkien, J. R. R. (John Ronald Reuel), 1892-1973.**

Resultado 12 de 18

[◀ Anterior](#) | [Listado](#) | [Siguiente ▶](#)

[Completo](#) [Ficha](#) [MARC](#)

LIBRO

El señor de los anillos

J. R. R. Tolkien ; traducción por Luis Domenech.

Buenos Aires : Minotauro, 2004.

3 vols. ; 17,5 cm.

ISBN: 9505470622

La biblioteca posee: v.1. La comunidad del anillo — v.2. *Las dos torres*.

Ubicación

- ❖ Vol. 1
[JN T649](#) Ej.1 [Sala Juvenil]
- ❖ Vol. 2
[JN T649](#) Ej.1 [Sala Juvenil]

Personas, entidades y obras relacionadas

- ❖ [Tolkien, J. R. R. \(John Ronald Reuel\), 1892-1973.](#) [18]
- ❖ [Domenech, Luis.](#) [7]

Temas

- ❖ [Literatura juvenil.](#) [1489]
- ❖ [Ciencia ficción.](#) [217]
- ❖ [Novelas de aventuras.](#) [828]

[Registro 073246](#) · Modificado: 14/05/2012

Buscar por palabras

Autor ▲
Título
Tema
ISBN/ISSN/etc
Cualquier campo ▼

contiene las palabras:

Buscar

Escriba una o más palabras; el sistema las combina automáticamente con un AND. Use * para buscar usando una raíz (p.ej. *fractal** encuentra *fractal*, *fractales*, *fractals*).

Explorar índices alfabéticos

Autor ▲
Título
Tema ▼

comienza por:

Explorar

Ingrese las primeras letras del autor o título que busca. Omite artículos iniciales en títulos.



3-Estructuras de datos

Buscar:

- que el almacenamiento y la manipulación de los datos sea directa
- que ayude a simplificar el programa

La estructura de los datos puede determinar la estructura del programa



Documentación del código

Explica qué hace el programa y cómo lo hace.

- Documentación **interna**: material descriptivo escrito directamente dentro del código
- Documentación **externa**: toda otra documentación que acompaña al programa





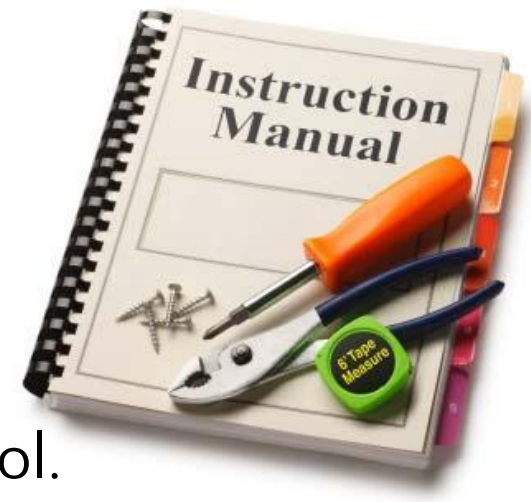
Documentación interna

Identifica el programa.

Describe algoritmos, estructuras de datos y flujo de control.

Usualmente ubicada al comienzo de cada componente. Incluye:

- Denominación del componente
- Quién lo ha escrito
- Dónde va ubicado en el diseño general del sistema
- Cuándo fue escrito/revisado/modificado
- Justificación del componente
- Uso que hace de las estructuras



Documentación interna y externa

- Recomendable en la documentación interna
 - Comentarios que acompañen el código
 - Nombres de variables significativas
 - Dar formato que ayude a la comprensión
- Incluir en la documentación externa
 - Descripción del problema
 - Descripción de los algoritmos elegidos
 - Descripción de los datos



Documentación interna - estándares

Header Comment Blocks

All source code files in the PEAR repository shall contain a "page-level" docblock at t above each class. Below are examples of such docblocks.

```
<?php

/* vim: set expandtab tabstop=4 shiftwidth=4 softtabstop=4: */

/**
 * Short description for file
 *
 * Long description for file (if any)...
 *
 * PHP version 5
 *
 * LICENSE: This source file is subject to version 3.01 of the PHP license
 * that is available through the world-wide-web at the following URI:
 * http://www.php.net/license/3_01.txt. If you did not receive a copy of
 * the PHP License and are unable to obtain it through the web, please
 * send a note to license@php.net so we can mail you a copy immediately.
 *
 * @category    CategoryName
 * @package     PackageName
 * @author      Original Author <author@example.com>
 * @author      Another Author <another@example.com>
 * @copyright   1997-2005 The PHP Group
 * @license     http://www.php.net/license/3_01.txt  PHP License 3.01
 * @version     SVN: $Id$
 * @link        http://pear.php.net/package/PackageName
 * @see         NetOther, Net_Sample::Net_Sample()
 */
```

```
* @link        http://pear.php.net/package/PackageName
* @see         NetOther, Net_Sample::Net_Sample()
* @since       File available since Release 1.2.0
* @deprecated  File deprecated in Release 2.0.0
*/

/*
 * Place includes, constant defines and $_GLOBAL settings here.
 * Make sure they have appropriate docblocks to avoid phpDocumentor
 * construing they are documented by the page-level docblock.
*/

/**
 * Short description for class
 *
 * Long description for class (if any)...
 *
 * @category    CategoryName
 * @package     PackageName
 * @author      Original Author <author@example.com>
 * @author      Another Author <another@example.com>
 * @copyright   1997-2005 The PHP Group
 * @license     http://www.php.net/license/3_01.txt  PHP License 3.01
 * @version     Release: @package_version@
 * @link        http://pear.php.net/package/PackageName
 * @see         NetOther, Net_Sample::Net_Sample()
 * @since       Class available since Release 1.2.0
 * @deprecated  Class deprecated in Release 2.0.0
*/

class Foo_Bar
{
}
```


Generación de documentación

```
/**
 * Returns an Image object that can then be painted on the screen.
 * The url argument must specify an absolute {@link URL}. The name
 * argument is a specifier that is relative to the url argument.
 * <p>
 * This method always returns immediately, whether or not the
 * image exists. When this applet attempts to draw the image on
 * the screen, the data will be loaded. The graphics primitives
 * that draw the image will incrementally paint on the screen.
 *
 * @param url an absolute URL giving the base location of the image
 * @param name the location of the image, relative to the url argument
 * @return the image at the specified URL
 * @see Image
 */
public Image getImage(URL url, String name) {
    try {
        return getImage(new URL(url, name));
    } catch (MalformedURLException e) {
        return null;
    }
}
```

getImage

```
public Image getImage(URL url,
                      String name)
```

Returns an `Image` object that can then be painted on the screen. The `url` argument must specify an absolute URL. The `name` argument is a specifier that is relative to the `url` argument.

This method always returns immediately, whether or not the image exists. When this applet attempts to draw the image on the screen, the data will be loaded. The graphics primitives that draw the image will incrementally paint on the screen.

Parameters:

`url` - an absolute URL giving the base location of the image.

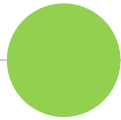
`name` - the location of the image, relative to the `url` argument.

Returns:

the image at the specified URL.

See Also:

`Image`



Generación de documentación

**Java™ 2 Platform
Standard Ed. 5.0**

[All Classes](#)

Packages

- [java.applet](#)
- [java.awt](#)
- [java.awt.color](#)
- [java.awt.datatransfer](#)

[java.math](#)

Classes

- [BigDecimal](#)
- [BigInteger](#)
- [MathContext](#)

Enums

- [RoundingMode](#)

[BigInteger\(String val\)](#)
Translates the decimal String representation of a BigInteger into a BigInteger.

[BigInteger\(String val, int radix\)](#)
Translates the String representation of a BigInteger in the specified radix into a BigInteger.

Method Summary

BigInteger	abs() Returns a BigInteger whose value is the absolute value of this BigInteger.
BigInteger	add(BigInteger val) Returns a BigInteger whose value is (this + val).
BigInteger	and(BigInteger val) Returns a BigInteger whose value is (this & val).
BigInteger	andNot(BigInteger val) Returns a BigInteger whose value is (this & ~val).
int	bitCount() Returns the number of bits in the two's complement representation of this BigInteger that differ from its sign bit.
int	bitLength() Returns the number of bits in the minimal two's-complement representation of this BigInteger, <i>excluding</i> a sign bit.
BigInteger	clearBit(int n) Returns a BigInteger whose value is equivalent to this BigInteger with the designated bit cleared.
int	compareTo(BigInteger val) Compares this BigInteger with the specified BigInteger.

2

Pruebas

Objetivos.
Defectos. Clasificación



Prueba de los programas

Se concentra en la **búsqueda de defectos** en los programas.

¿Por qué falla el software?

- ◉ Especificación **errónea** o requerimientos **omitidos**
- ◉ Requerimientos **imposibles** de implementar dado el hw y sw con los que se cuenta
- ◉ El diseño del **sistema** es defectuoso
- ◉ El diseño del **programa** es defectuoso
- ◉ El **código** tiene **errores**



Objetivo de la prueba

Se prueba para **demostrar la existencia de defectos**.

La prueba es destructiva, se considera exitosa solamente si encuentra defectos o si se producen fallas como consecuencia de los procedimientos de prueba.

La visión del programador es opuesta: busca demostrar que su código es correcto. Esto va en contra del espíritu de las pruebas.



Tipos de defectos-1

- **Algorítmicos:** el algoritmo o la lógica de un componente no producen la salida apropiada para la entrada dada, por errores en los pasos del procesamiento
 - Bifurcar antes o después de tiempo
 - Probar una condición errónea
 - Olvidar inicializar variables o constantes de un bucle
 - Olvidar probar una condición particular
 - Comparar variables de tipos de datos inapropiados
- **De sintaxis:** en las estructuras del lenguaje de programación.
- **De computación y de precisión:** implementación erróneas de fórmulas o cálculos sin la exactitud necesaria



Tipos de defectos-2

- ◉ **De documentación:** cuando esta no se corresponde con lo que el programa verdaderamente hace
- ◉ **Por estrés o sobrecarga:** se sobrepasan las estructuras
- ◉ **De capacidad o límites:** se deteriora el desempeño del sistema por superar sus límites especificados
- ◉ **De sincronización o coordinación:** coordinación inadecuada de procesos que deben ejecutar de forma simultánea
- ◉ **De rendimiento o desempeño:** el sistema no opera con el desempeño (velocidad, seguridad, exactitud, confiabilidad) esperado
- ◉ **De recuperación:** la respuesta a una falla no es la esperada



Visiones de los objetos de prueba

- Caja **cerrada** o **negra**
 - El objeto se analiza desconociendo el contenido
 - Las pruebas consisten en alimentar la “caja negra” con entradas y observar cuáles son las salidas
- Caja **abierta** o **blanca**
 - Con la visión de caja cerrada no se puede elegir casos de prueba representativos
 - Se utiliza la estructura del objeto para probar de diversas maneras
 - Por ejemplo: todos los caminos posibles del código



¿Cómo probar ...

...carga tolerada?

...desgaste de cubiertas?

...consumo de combustible?

...alcance de luces?

...vida útil de la batería?

...seguridad ante accidentes?





Organización de las pruebas

1. De módulo, de componente, o de unidad
 - Verificar funcionamiento del componente con los tipos de entrada esperados, estructuras de los datos, lógica y condiciones de límites para los datos
2. De integración
 - Verificar el funcionamiento conjunto de los componentes del sistema
3. Funcional
 - Determinar si las funciones descritas en la especificación son realmente ejecutadas por el sistema integrado



Organización de las pruebas

4. De rendimiento o desempeño
 - Comparar el sistema con los requerimientos de hardware y software.
5. De aceptación
 - En conjunto con el cliente se prueba el sistema contra las especificaciones de requerimientos
6. De instalación
 - Se prueba una vez más, para garantizar que una vez instalado funcione como debe hacerlo



Casos de prueba

Casos de prueba: conjunto particular de casos de entrada a ser utilizado en la prueba de un programa.

La selección de los casos de prueba se hace para utilizar al menos un **enfoque**:

- Comprobación de sentencias
- Pruebas de la ramificación
- Comprobación del camino
- ...



Minuciosidad de la prueba

- **Comprobación de sentencias:** ejecutar al menos una vez cada sentencia del componente
- **Pruebas de la ramificación:** cada rama de cada decisión en el código se elige al menos una vez
- **Comprobación del camino:** cada camino distinto formado en el código se ejecuta por lo menos una vez
- ...



Planificación de la prueba

Pasos

1. Determinación de los objetivos de la prueba
2. Diseño de los casos de prueba
3. Preparación escrita de los casos de prueba
4. Verificación de los casos de prueba
5. Ejecución de los casos de prueba
6. Evaluación de los resultados



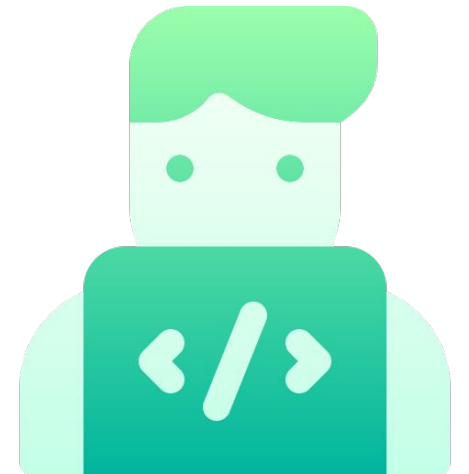
Equipo de prueba

Los programadores están demasiado familiarizados con la estructura e intención de la implementación y pueden tener dificultades reconociendo errores y diferencias con los requerimientos.

Idealmente el equipo de pruebas es independiente del equipo de implementación.

Son profesionales con experiencia y se involucran en esta tarea desde el comienzo del proyecto.

- Organizar las pruebas
- Diseñar las pruebas
- Ejecutar las pruebas





Resumen

Implementación

Estructura.
Documentación.

Pruebas

Tipos de defectos. Clasificación.
Visión de los objetos de prueba.
Organización de las pruebas.
Casos de prueba.
Planificación de la prueba.
Equipo de prueba.



RESUMEN DE LA UNIDAD.2

- ◉ ¿Qué es la Ingeniería de Software?
- ◉ Proceso de resolución de problemas
- ◉ Participantes y roles en el proceso
- ◉ Sistemas. Software producto y herramienta
- ◉ Proceso. Flujos. Modelos.
- ◉ Ciclos de vida
- ◉ Ingeniería de Requerimientos



RESUMEN DE LA UNIDAD.2

- ◉ Diseño. Descomposición
- ◉ Diseño arquitectónico. Patrones.
- ◉ Modularidad. Abstracción. Acoplamiento. Cohesión.
- ◉ Implementación. Codificación.
- ◉ Buenas prácticas. Documentación
- ◉ Pruebas. Tipos de pruebas. De proceso. De producto.



Bibliografía



- *Ingeniería del software. Un enfoque práctico* – R. Pressman
Estrategias de prueba de software. – Capítulo 17 (Capítulo 22)
Prueba de aplicaciones convencionales. – Capítulo 18 (Capítulo 23)
- *Ingeniería de software . Teoría y Práctica* – S. L. Pflegger
Capítulo 7 – Escribiendo los programas.

Template: www.slidescarnival.com

Mg. M. Clara Casalini. – Dr. Axel Soto – 2025

Introducción a la ingeniería de Software – Ingeniería en Sistemas de Información

Departamento de Ciencias e Ingeniería de la Computación – Universidad Nacional del Sur